

CLAUDE.md Behavior Templates

Copy-Paste Configurations for Claude Code (July 2026 edition)

Pick 3-5 templates that solve your biggest pain points and paste them into your project's CLAUDE.md. Claude loads CLAUDE.md at session start and treats it as project policy.

Two rules before you start:

1. **Keep it under ~200 lines.** Longer files get progressively ignored. Split overflow into `.claude/rules/` files.
2. **You don't have to write everything.** Since 2026, Claude Code keeps **auto memory** - its own notes on build commands, test invocations, and project insights, stored per-project and loaded alongside CLAUDE.md. Check it with `/memory`. Curate CLAUDE.md for POLICY (rules, conventions, boundaries); let auto memory accumulate FACTS (how the build works). If you see Claude re-discovering the same fact every session, promote it into CLAUDE.md.

Recommended Starter Pack

1. Reduce Hallucinations

```
<investigate_before_answering>
Never speculate about code you have not opened.
If I reference a specific file, you MUST read that file before answering.
Investigate and read relevant files BEFORE answering questions about
the codebase. Give grounded answers based on actual code inspection.
</investigate_before_answering>
```

2. Maximize Parallel Execution

```
<use_parallel_tool_calls>
If you intend to call multiple tools with no dependencies between
them, make all independent calls in parallel. Reading 3 files = 3
parallel calls. If a call depends on a previous result, run it
sequentially. Never guess missing parameters.
</use_parallel_tool_calls>
```

3. Context Window Management

```
<context_management>
Your context window is compacted automatically as it approaches its
limit. Do not stop tasks early due to token budget concerns. As you
approach the limit, save progress and state to memory or a
progress.txt file. Complete tasks fully.
</context_management>
```

Behavior Control

4. Hyper-Proactive Mode

```
<default_to_action>
By default, implement changes rather than only suggesting them. If my
intent is unclear, infer the most useful action and proceed, using
tools to discover missing details. Implement good ideas you find
along the way - I review afterward.
</default_to_action>
```

5. Conservative Mode

```
<do_not_act_before_instructions>
Do not change files unless clearly instructed. When intent is
ambiguous, default to research and recommendations. Ask before
significant architectural decisions.
</do_not_act_before_instructions>
```

These two are opposites - pick ONE. Proactive suits fast reviewers and Auto-mode workflows; Conservative suits learning and critical systems. Whichever you pick, your permission rules (deny -> ask -> allow) remain the hard fence.

6. Complete Work Systematically

```
<complete_work_systematically>
Plan the work clearly. Continue systematically until complete. Don't
leave functions half-implemented. Finish each component fully before
moving to the next. Don't run out of context with significant
uncommitted work.
</complete_work_systematically>
```

Code Quality

7. Prevent Over-Engineering

```
<avoid_overengineering>
Only make changes that are directly requested or clearly necessary.
A bug fix doesn't need surrounding cleanup. Don't add error handling
for scenarios that can't happen - validate at system boundaries only.
No helpers or abstractions for one-time operations. No hypothetical
future requirements. Minimum complexity for the current task.
</avoid_overengineering>
```

8. Real Solutions, Not Test-Passing Hacks

```
<write_real_solutions>
Implement solutions that work for all valid inputs, not just test
cases. Do not hard-code values to pass tests. Tests verify
correctness - they don't define the solution. If a test is wrong,
tell me instead of working around it.
</write_real_solutions>
```

Operations

9. Compact Instructions (survives /compact)

```
## Compact Instructions
When compressing, preserve in priority order:
1. Architecture decisions (NEVER summarize)
2. Modified files and their key changes
3. Current verification status (pass/fail)
4. Open TODOs and rollback notes
5. Tool outputs (keep pass/fail only)
```

10. State Tracking for Long Projects

```
<state_tracking>
Maintain state across sessions:
1. progress.txt - what's completed, what's next
2. tests.json - test status (tests only grow, never shrink)
3. Frequent commits with clear messages
At session start: review progress.txt, check recent git log, run
tests before new features.
</state_tracking>
```

11. Explain Everything (Learning Mode)

```
<explain_your_work>
After any task involving tool use, summarize: what you did, why that
way, and the outcome.
</explain_your_work>
```

12. Frontend Aesthetics

```
<frontend_aesthetics>
Avoid generic "AI slop" design. Distinctive typography (no default
Arial/Inter), a committed color theme with CSS variables, purposeful
motion, layered backgrounds. Avoid cliched purple-gradient-on-white.
</frontend_aesthetics>
```

Choosing Your Configuration

Working style	Templates
Fast & autonomous (Auto mode)	#4 + #2 + #1
Careful & controlled	#5 + #1 + #11
Long multi-session projects	#3 + #10 + #6 + #9
Frontend work	#12 + #7 + #4
Learning	#11 + #5 + #1

Pro Tips

1. **Explain motivation, not just tasks** - "Build a calendar as the central hub all notes integrate with" beats "build a calendar component."
2. **Bootstrap with `/init`**, then curate - don't write from scratch.
3. **Check `/memory` monthly** - promote recurring auto-memory facts into CLAUDE.md; delete rules the model now does by default.
4. **Commit frequently** - history is context Claude can mine.
5. **Wrap critical rules in XML tags** - tagged content gets more attention in longer files.
6. **A/B test your policy** - rename CLAUDE.md off, run the same prompt, diff the results. If there's no difference, your rules match model defaults; sharpen them.

(c) 2026 AIA Copilot - Claude Code Training

The Behavioral Foundation - Four Lines First

Before any template below, start your CLAUDE.md with these four behavioral lines (derived from Andrej Karpathy's January 2026 failure-mode diagnosis; ~100K-star community standard):

1. Don't assume. Don't hide confusion. Surface tradeoffs.
2. Minimum code that solves the problem. Nothing speculative.
3. Touch only what you must. Clean up only your own mess.
4. Define success criteria. Loop until verified.

Why behavior beats rule lists: rules like "use strict mode" apply to one stack; "don't assume" applies to everything. Keep the rest of your file to what the agent can't infer: build commands, invisible conventions, watch-outs. Litmus test for every other line: would removing it cause a mistake? If not, cut it. (Star counts and install paths drift - the four lines are stable.)